

AI Career Navigator — Project Brief

1. Project Idea

AI Career Navigator is an NLP and Machine Learning application that helps users discover suitable career opportunities based on their CV.

The system will:

- Read and analyze a user's CV.
- Extract skills, education, experience, and other relevant information.
- Analyze available job descriptions.
- Predict how well the candidate matches each job.
- Identify missing skills for specific jobs.
- Display suitable job opportunities through a Streamlit interface.

2. Dataset

The main training dataset will be the **Candidate Matching Synthetic Dataset** from Hugging Face:

[Hugging Face — Candidate Matching Synthetic Dataset](#)

The dataset will be used to create candidate-job examples and train the matching model.

Because the dataset is synthetic, the project should be presented as a **prototype/research project**, not as a real-world hiring prediction system.

2.1 Source Tables

The dataset is distributed as three separate tables (stored as Parquet files in the `resumes/`, `jobs/` and `matches/` folders of the repository). The `embeddings/` folder is not used in the core project.

Table	Rows	Columns
Resumes	10,000	resume_id, role, seniority, years_experience, industry, education, skills, summary, experience_bullets
Jobs	2,500	job_id, job_title, seniority, industry, must_have_skills, nice_to_have_skills, description, responsibilities, requirements
Matches	2,500	job_id, relevant_resume_ids (about 30 matching resumes per job)

The tables cannot be used for training directly, because resumes and jobs are separate entities and the matches table stores its links as a list inside a single cell.

2.2 Merged Training Dataset (Candidate-Job Pairs)

The three tables are merged into **one CSV file of candidate-job pairs**, where each row is one resume paired with one job:

```

Matches table
↓
Explode lists → one row per (job, resume) pair      (~75,000 pairs, label = 1)
+
Sampled non-matching pairs                          (~75,000 pairs, label = 0)
↓
Merge resume details (on resume_id) + job details (on job_id)
↓
candidate_job_pairs.csv    (~150,000 rows)

```

- Resume columns are prefixed with `r_` and job columns with `j_` (both tables contain `seniority` and `industry`).
- List columns (skills, bullets, responsibilities, requirements) are stored as `|`-separated text in the CSV and split back into lists when loaded.
- The original resume and job text is not modified.

2.3 Target Variable

The target is `label1`:

- `1` = the resume matches the job
- `0` = the resume does not match the job

In the source dataset, a match is defined by a skill rule: **at least 60% of the job's must_have_skills appear in the resume's skills**. The dataset only contains matches, so the **non-matching pairs (label = 0) are created by the project** by pairing each job with resumes that are not in its relevant list. The final dataset is balanced (50% / 50%).

Non-matching pairs that accidentally satisfy the 60% rule are checked and removed. To make the task harder than random pairing, **hard negatives** (resumes with about 30–59% must-have skill overlap) can also be included.

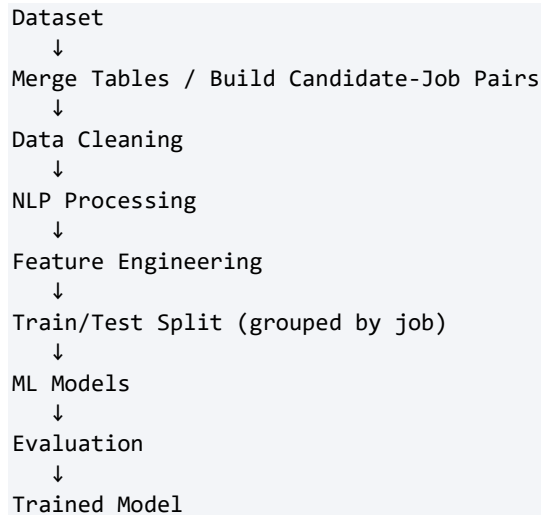
2.4 Data Notes and Limitations

- **Label is skill-based:** because the label comes from skill overlap, skill-overlap features will make the task easy for the models. Results should be reported with and without the direct overlap features, and this limitation should be stated clearly.
- **Repeated entities:** the same resume and job appear in many pairs (10,000 unique resumes and 2,500 unique jobs in about 150,000 rows). The train/test split must be grouped by `job_id` so no job appears in both sets.
- **Balanced classes are artificial:** in real job search, most CV-job pairs do not match, so real-world performance can differ.
- **Limited fields:** jobs have no required years of experience and no education requirement, only `seniority`.
- **Generic text:** summaries, descriptions, responsibilities, requirements and experience bullets are mostly template text, so the most informative columns are the skills, seniority, role/title and industry.
- **Small skill vocabulary:** the dataset contains 73 unique skills, so skills extracted from real CVs and live job postings need to be mapped to this vocabulary.

3. Project Has Two Main Phases

Phase 1 — Training

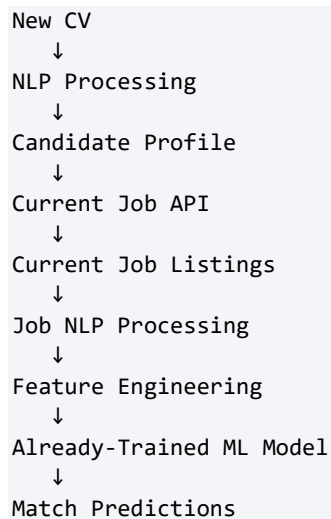
Use the existing dataset to train the ML model.



The model is trained **once** using the training dataset.

Phase 2 — New User

When a new user opens the application:



The model is **not retrained for every user**.

4. NLP

NLP will be used to process both CVs and job descriptions.

CV Processing

Extract information such as:

- Skills
- Education
- Years of experience
- Job titles
- Experience descriptions
- Relevant keywords

Job Processing

Extract:

- Required skills
- Job title
- Required experience
- Education requirements
- Job description
- Seniority

Note: the training dataset provides job skills, title, seniority and industry directly, but not required years of experience or education requirements. These fields are extracted only for live job listings (and used for display or skill-gap analysis), not as training features.

Possible NLP techniques:

- Text cleaning
- Tokenization
- Stop-word removal
- TF-IDF
- Keyword/skill extraction (mapped to the dataset's skill vocabulary)
- Text similarity

Optional: Sentence-BERT can be added later for semantic similarity.

5. Feature Engineering

After NLP processing, candidate and job information will be converted into ML features.

Possible features:

- Skill similarity
- Number of matching skills
- Missing skills count
- Experience compatibility (candidate `years_experience` compared with the range implied by the job's seniority: Junior 0–2, Mid 2–6, Senior 6–12 years)
- Seniority compatibility (candidate `seniority` vs job `seniority`)
- Role/title similarity and industry match
- Text similarity
- Other candidate-job compatibility features

Education compatibility is not available as a training feature because the jobs in the dataset have no education requirement. The candidate's education is still extracted from the CV for the candidate profile.

Example:

```
Candidate Skills
      +
Job Requirements
      ↓
Feature Engineering
      ↓
ML Features
```

Feature engineering happens **before the train/test split** when creating the training examples, while the exact same feature-generation process is reused during inference.

6. Machine Learning

Several supervised ML models can be tested:

Logistic Regression

Baseline model for predicting match/non-match.

SVM

Useful for classification with high-dimensional text-related features.

Random Forest

Useful for handling a mixture of numerical and categorical/engineered features.

XGBoost

A stronger tree-based model that can be tested against the other models.

The models will be evaluated using appropriate classification metrics such as:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

The train/test split is grouped by `job_id`. Because the label is derived from skill overlap, models are compared both with and without the direct skill-overlap features.

The project will compare the models based on their measured evaluation results rather than assuming one model will perform best beforehand.

7. Current Job Data

After training, the application needs **current job opportunities** to match against the new user's CV.

Instead of maintaining a database initially, the Streamlit application can call a public job API and load the returned jobs into a Pandas DataFrame.

Job API Options

Option 1 — Arbeitnow

[Arbeitnow Job Board API](#)

Useful for getting job listings in JSON format.

Option 2 — Adzuna

[Adzuna API Documentation](#)

Adzuna provides job-search endpoints and requires API credentials (app_id and app_key).

Option 3 — Remote

[Remote Public Jobs API](#)

Useful if the project focuses on remote jobs. Remote's public listings are delayed by 24 hours.

Recommended approach for the MVP: start with one API only. The API should return job information such as:

- Job title
- Company
- Location
- Description
- Required skills/tags
- Job URL
- Other available metadata

The project does **not** need a database for this initial version.

8. New User CV

The user uploads their CV through Streamlit.

```
User uploads CV
      ↓
Extract text
      ↓
NLP processing
      ↓
Candidate Profile
```

The candidate profile can contain:

```
Skills
Education
Experience
Job Titles
Summary
```

9. Matching

For every current job:

```
Candidate Profile
+
Job Profile
↓
Feature Engineering
↓
Trained ML Model
↓
Match Prediction
```

The application can then display suitable jobs to the user.

Example:

```
Software Engineer
Company: XYZ
Match: 87%
Missing Skills: Docker, AWS
```

The exact displayed "match %" should be based on the model's calibrated probability or another clearly defined score, rather than treating raw classifier outputs as percentages automatically.

10. Skill Gap Analysis

Skill-gap analysis is a separate component from the ML prediction.

```
Candidate Skills
vs
Job Required Skills
↓
Common Skills
Missing Skills
```

Example:

```
Candidate:
Python, SQL, Pandas, Machine Learning

Job Requires:
Python, SQL, Pandas, Machine Learning, Docker, AWS

Missing:
Docker
AWS
```

This allows the application to tell the user **why they may not fully match a particular job**.

11. Streamlit Application

The final application will use **Streamlit** as the frontend and application layer.

No separate React frontend or FastAPI backend is required for the initial version.

Possible UI:

AI Career Navigator

Upload your CV

[Choose File]

[Analyze CV]

Candidate Profile

Skills: Python, SQL, ML

Experience: 1 year

Education: Computer Science

Recommended Jobs

1. Data Analyst

Match: XX%

Missing Skills: Power BI

2. ML Intern

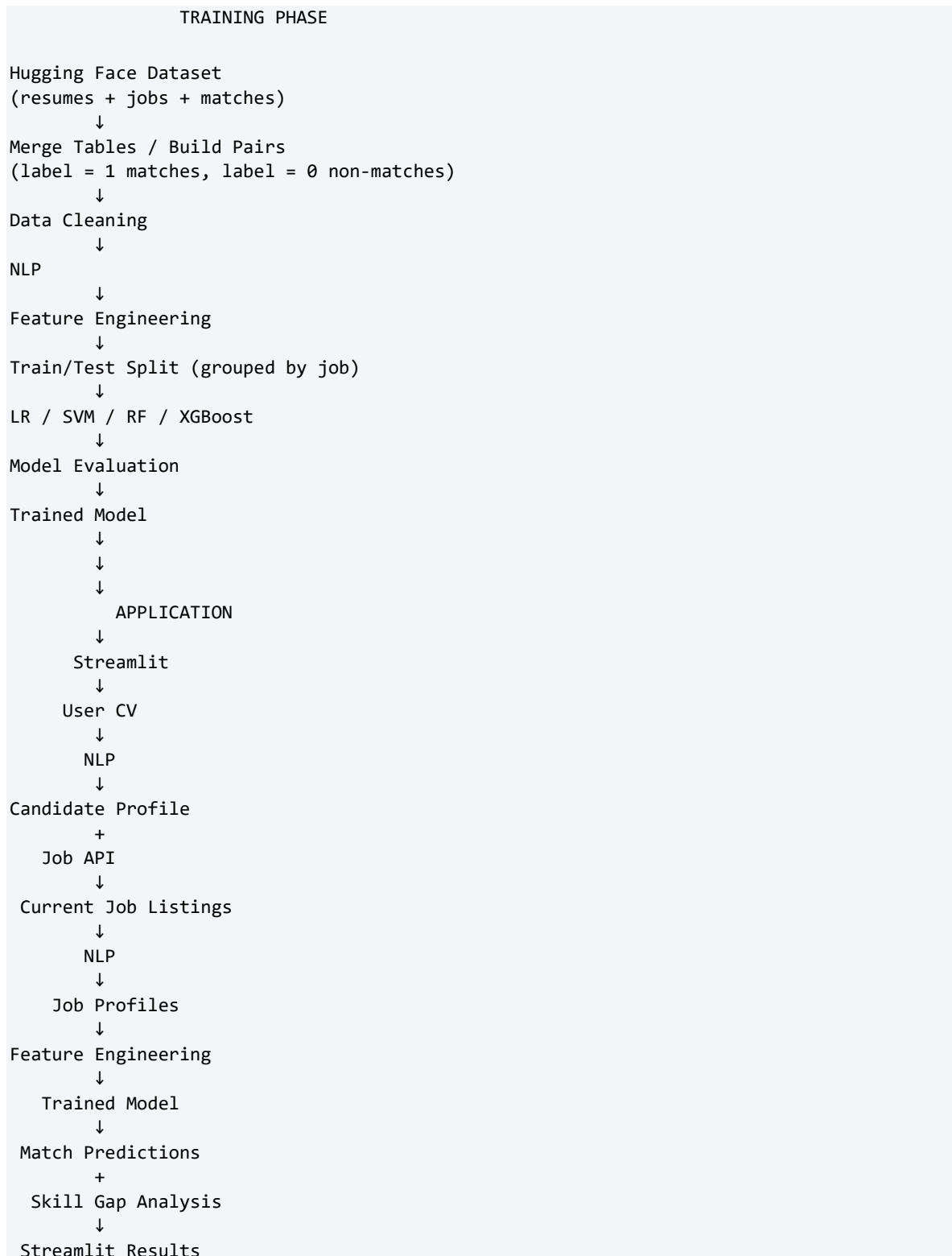
Match: XX%

Missing Skills: Docker

Skill Gap Analysis

12. Project Architecture

The overall system is:



13. One Jupyter Notebook

The development/training work will be organized in **one Jupyter Notebook**.

Suggested structure:

1. Imports
2. Load Dataset (resumes, jobs, matches)
3. Explore Dataset
4. Data Cleaning
5. NLP Processing
6. Feature Engineering
7. Create Training Dataset (merge into candidate-job pairs, label = target)
8. Train/Test Split (grouped by job_id)
9. Logistic Regression
10. SVM
11. Random Forest
12. XGBoost
13. Model Evaluation
14. Select / Save Model
15. Test on New CV
16. Collect Current Jobs
17. Match CV Against Jobs
18. Skill Gap Analysis

The notebook will contain the main development and ML work, while **Streamlit will provide the final user interface.**

14. Core Project — MUST Have

The minimum complete project should include:

Data

- Hugging Face candidate-job dataset (resumes, jobs and matches tables).
- Merged candidate-job pairs dataset with a binary `label` target.

NLP

- CV text processing.
- Job description processing.

- Skill extraction.
- Text similarity.

Feature Engineering

- Skill similarity.
- Experience compatibility (based on years of experience vs job seniority).
- Seniority compatibility.
- Other candidate-job features.

Machine Learning

- Logistic Regression.
- SVM.
- Random Forest.
- XGBoost.
- Model evaluation/comparison (grouped train/test split, with and without skill-overlap features).

Real-World Component

- Current job data through a job API.
- Candidate CV matched against current jobs.

Application

- Streamlit.
- CV upload.
- Candidate profile.
- Job matching results.
- Skill-gap results.

15. BONUS Features — ONLY If There Is Enough Time

These features are optional and should not delay the core project.

Bonus 1 — Sentence-BERT

Replace/basic TF-IDF similarity with semantic embeddings.

```
CV → Sentence-BERT → Embedding
Job → Sentence-BERT → Embedding
                        ↓
                    Similarity Score
```

Bonus 2 — Explainable AI

Use SHAP or similar techniques to show which features contributed to a prediction.

Bonus 3 — Career Category Prediction

Predict a possible career category based on the user's skills and CV.

Bonus 4 — Learning Recommendations

For missing skills, recommend topics/resources the user could study.

Bonus 5 — Job Market Analytics

Use current job data to show:

- Most requested skills
- Popular job categories
- Common technologies
- Job distribution

Bonus 6 — Richer Streamlit Dashboard

Add charts, filters, statistics, and visual career insights.

Bonus 7 — Hard Negatives

Add harder non-matching pairs (partial skill overlap) to make the training task more realistic.

16. Final Technology Stack

```
Python
|
|— Pandas / NumPy
|
|— Scikit-learn
|
|— XGBoost
|
|— NLP libraries
|
|— Hugging Face Dataset
|
|— Job API
|
|— Jupyter Notebook
|
|— Streamlit
```

No database, React frontend, or FastAPI backend is required for the initial version.

17. Final Project Description

AI Career Navigator is an NLP and supervised machine-learning application built with Streamlit. It analyzes a user's CV, extracts relevant career information, compares the candidate with current job opportunities obtained through a job API, predicts job compatibility using trained ML models, and identifies skill gaps between the candidate and potential jobs.